ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
Funciones y Arreglos

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Funciones en C

- Encapsula código.
- Permite reutilizar código.
- Se pueden compilar por separado.
- Retorna siempre un tipo de dato específico.

Funciones en C

- Se puede separar interfaz (prototipo) de implementación.
- **Interfaz (prototipo):** Sólo define existencia y formato de función. Suficiente para ser invocada.

C

```c
//tipo_retorno funcion(parametros con o sin nombre);
float potencia(float a, float b);
//....

calculo = x * potencia(y, 2.5);
```

- **Implementación:** Define implementación de código de función.

C

```c
/*tipo_retorno funcion(parametros con nombre) {
*       Implementacion...
}*/

float potencia(float a, float b) {
        return power(a,b);
}
```

Funciones en C

- Si se utiliza solamente la implementación, está tiene que estar definida **antes** de la función que la invoca.
- Si se utiliza el prototipo, es éste el que debe estar defindo **antes** de la función que la invoca.
- Prototipo + Implementación: libertad de ubicación de la implementación.

Funciones en Python

- **Python** sólo permite implementación, no prototipos.

Python

```
def nombre_funcion(parametros):
    #...Implementacion
    return <expresion>
```

```
def potencia(x, y):
    print x, y
    return x**y

y = 2
calculo = 5 * potencia(y, 2.5)
print calculo
```

```
2 2.5
28.2842712475
```

Macro

Una macro es un valor o conjuntos de instrucciones constantes que se definen a nivel de preprocesamiento. No es una variable y no puede ser modificada.

C

```c
#include<stdio.h>
#define MACRO 10

int main(){
        int a = 5;
        printf("%"s\n", a*MACRO);
}
```

Constante

Una variable constante es aquella que se declara con el keyword **const**, e implica que ésta no puede ser modificada. Tiene todo el resto de las propiedades de una variable común.

C

```c
#include<stdio.h>

int main(){
        const int a = 5;
        a = 10; //¿Cómo actúa el compilador?
        printf("%"d\n", a);
}
```

Ámbito (Scope)

Las funciones (includa main) definen el alcance de las variables, llamado **ámbito**, vale decir, el segmento en el cual existen y pueden ser usadas → entre { y }.

C

```c
#include<stdio.h>

void saludo(){ // var i sirve sólo dentro de función saludo.
        int i = 5;
        printf("Hola alumno numero %d", i);
}

int main(){// var a sirve sólo dentro de función main.
        int a = 10;
        saludo();
        printf("%"d\n", a);
        return 0;
}
```

Variable local

Una variable local, es aquella que está definida dentro de un ámbito de terminado, vale decir, una función y/o estructura de control.

C

```c
#include<stdio.h>

int main(){// var a sirve sólo dentro de función main.
        int a = 10;

        if(a==10){
                int b = 1;
                printf("%d %d\n", a, b);
        }
        // ¿Qué pasa con la variable b?
        printf("%d %d\n", a, b);

        return 0;
}
```

Variable Global

Una variable global, es aquella que está definida fuera de cualquier ámbito. Puede ser llamada, modificada y utilizada desde cualquier ámbito, siempre y cuando no exista una variable local con el mismo nombre.

C

```c
#include<stdio.h>

int a=1; //global
int b=2; //global

int main(){
        int a = 10; //local
        b++;
        printf("%d %d\n", a, b);//¿Qué imprime?
        return 0;
}
```

En python, no se pueden modificar variables globales dentro de una función.

Arreglos y Strings

- **Arreglo:** Tipo estructurado consistente en conjunto homogéneo y ordenado de elementos.
- Se identifican por su posición relativa mediante un *índice* encerrado con paréntesis cuadrados (parte de índice 0).

```
//Arreglos estáticos (se liberan al terminar
// el ámbito donde fueron declaradas):
int foo[10]; //10 enteros creados en foo.
foo[3] = 7; //A cuarto elemento se asigna un 7.
```

- **String:** Arreglo de caracteres.

```
// Los string se delimitan con caracter '\0'
// (requiere espacio de 1 char):
char nombre[10] = "Juan"; //String de largo máximo 10.
nombre[1] = 'e'; //Qué hace?
printf("%s\n", nombre);
```

- Veremos más sobre strings la próxima clase.

Arreglos Multidimensionales e Inicialización

Al igual que con las listas de Python, pueden definir arreglos multidimensionales.
Además los arreglos pueden ser inicializados utilizando { y }.

C

```
#include<stdio.h>

int main(){
        char nom[10] = "Camilo";
        char a[5][20] = {"Juan", "Francisco", "Pedro"};
        int b[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}}, i, j;

        for(i=0;i<3;i++){
                for(j=0;j<3;j++) //¿Qué pasó con las llaves?
                        printf("%d ", b[i][j]);
                printf("\n");
        }
}
```

Recorrer un arreglo

Podemos utilizar cualquiera de las estructuras de control iterativas para recorrerlos.

C

```
#inclede<stdio.h>

int main(){
        int i;
        float a[5];

        for(i=0;i<5;i++){
                float val; //¿Cuántas veces nace y muere val?
                scanf("%f", &val);
                a[i]=val;
        } //¿Cómo reducir este código?

}
```

Recorrer un arreglo

Practiquen con los siguientes ejercicios:

- Calcular el factorial de un número dado.
- Ordenar de mayor a menor un arreglo de una dimensión.
- Encontrar el mayor elemento en un arreglo de dos dimensiones.
- Calcular el promedio de los elementos en un arreglo bidimensional.
- Imprimir en orden inverso los elementos de un arreglo.
- Imprimir todas las letras mayúsculas y minúsculas del alfabeto (ver Código ASCII).